# Agentes de Inteligencia Artificial, LangChain y LangGraph

Investigación para el proyecto "Agente para Compras Inteligentes"
Materia: Inteligencia Artificial

**Alumno(a):** _____

**Profesor(a):** _____

**Fecha:** _____

# 1. Introducción

Los modelos de lenguaje de gran tamaño (LLM, *Large Language Models*) como Llama, GPT o Claude son capaces de entender y generar texto. Por sí solos, sin embargo, solo responden con lo que aprendieron durante su entrenamiento: no consultan información actual, no ejecutan cálculos exactos y no realizan acciones en el mundo. Los **agentes** resuelven esta limitación porque combinan un LLM con **herramientas** y un **ciclo de decisión**, de modo que el modelo puede razonar, actuar y aprender del resultado de sus acciones.

Este documento explica qué es un agente, cómo funcionan los agentes basados en LLM, qué son LangChain y LangGraph, cómo se relacionan, y en qué se diferencia un sistema de un solo agente de uno multiagente. Al final se relaciona todo con el proyecto desarrollado.

# 2. ¿Qué es un agente?

En Inteligencia Artificial, un **agente** es una entidad que **percibe** su entorno y **actúa** sobre él para alcanzar un objetivo (Russell y Norvig). Un termostato que mide la temperatura y enciende la calefacción es un agente muy simple; un auto autónomo es uno muy complejo.

## 2.1 Componentes de un agente

- **Percepción:** la información que recibe (en un agente con LLM, el mensaje del usuario y los resultados de las herramientas).
- **Razonamiento o decisión:** el mecanismo que elige qué hacer (en un agente con LLM, el propio modelo).
- **Acciones:** lo que puede hacer (en un agente con LLM, sus herramientas: buscar, calcular, consultar una API).
- **Memoria o estado:** lo que recuerda de lo que ya hizo y observó.

### 2.2 Tipos clásicos de agentes

| Tipo | Cómo decide | Ejemplo |
|---|---|---|
| Reactivo simple | Reglas "si pasa X, haz Y", sin memoria. | Termostato. |
| Basado en modelo | Mantiene un estado interno del mundo. | Robot aspiradora que mapea la casa. |
| Basado en objetivos | Elige acciones que lo acercan a una meta. | Navegador GPS. |
| Basado en utilidad | Elige la acción que maximiza un beneficio. | Sistema que recomienda la mejor compra. |
| Que aprende | Mejora con la experiencia. | Sistemas de recomendación de video. |

## 3. Agentes basados en LLM

Un agente basado en LLM usa el modelo de lenguaje como "cerebro". El modelo recibe instrucciones, el historial de la conversación y la descripción de las herramientas disponibles. Con eso decide si debe usar una herramienta (y con qué parámetros) o si ya puede responder.

### 3.1 Uso de herramientas (*tool calling*)

Los LLM modernos soportan **tool calling**: el programador describe cada herramienta (nombre, descripción y parámetros) y, cuando el modelo considera que necesita una, en lugar de responder con texto devuelve una **petición estructurada**, por ejemplo `buscar_productos(consulta="audífonos")`. El programa ejecuta la función real y le devuelve el resultado al modelo. El LLM nunca ejecuta código: solo *decide* qué ejecutar.

### 3.2 El patrón ReAct

ReAct (*Reasoning* + *Acting*) fue propuesto por Yao et al. (2022). El agente alterna entre **razonar** qué necesita, **actuar** usando una herramienta y **observar** el resultado, y repite el ciclo hasta tener la información suficiente para dar una respuesta final. Este patrón reduce las "alucinaciones", porque el modelo basa su respuesta en datos que obtuvo de las herramientas y no solo en su memoria.




Figura 1. Ciclo del patrón ReAct.

## 4. LangChain

**LangChain** es un framework de código abierto (Python y JavaScript) para construir aplicaciones con LLM. Ofrece piezas reutilizables y una interfaz común para trabajar con distintos proveedores de modelos:

- **Modelos de chat:** una misma interfaz para Groq, Ollama, OpenAI, Anthropic, etc. Cambiar de proveedor implica cambiar muy poco código.
- **Mensajes:** `SystemMessage`, `HumanMessage`, `AIMessage` y `ToolMessage` representan la conversación.
- **Herramientas:** el decorador `@tool` convierte una función de Python en una herramienta que el LLM puede usar; su *docstring* sirve de descripción.
- **Prompts, cadenas y recuperación de documentos (RAG)**, entre otros.

Las "cadenas" de LangChain funcionan muy bien para flujos lineales (paso 1 → paso 2 → paso 3), pero un agente necesita **ciclos** y **decisiones**, y para eso surgió LangGraph.

## 5. LangGraph

**LangGraph** es una librería del mismo equipo de LangChain para construir agentes como **grafos con estado**. Sus conceptos principales son:

| Concepto | Qué es |
|---|---|
| **Estado** | La información compartida entre los pasos del grafo. Normalmente es un `TypedDict`. Los *reductores* como `add_messages` indican cómo combinar los cambios (por ejemplo, agregar mensajes en lugar de reemplazarlos). |
| **Nodos** | Funciones de Python que reciben el estado, hacen un trabajo (llamar al LLM, ejecutar herramientas) y devuelven una actualización del estado. |
| **Aristas** | Conexiones entre nodos que indican el orden de ejecución. |
| **Aristas condicionales** | Una función decide a qué nodo ir según el estado. Por ejemplo, si el LLM pidió una herramienta va al nodo de herramientas; si no, termina. |
| **START / END** | Los puntos de inicio y fin del grafo. |
| **Ciclos** | A diferencia de una cadena lineal, el grafo puede regresar a un nodo anterior. Así se implementa el ciclo ReAct. |

LangGraph también ofrece **streaming** (ver cada paso en cuanto ocurre), **persistencia o memoria** entre conversaciones, intervención humana (*human-in-the-loop*) y componentes listos como `ToolNode`, que ejecuta automáticamente las herramientas que pide el LLM.

### 5.1 Relación entre LangChain y LangGraph

| | LangChain | LangGraph |
|---|---|---|
| Propósito | Piezas para trabajar con LLM (modelos, mensajes, herramientas). | Orquestar el flujo de un agente como grafo con estado. |
| Tipo de flujo | Principalmente lineal (cadenas). | Grafos con ciclos y condiciones. |
| Relación | Son complementarios: LangGraph usa los modelos, mensajes y herramientas de LangChain. En este proyecto LangChain aporta `@tool`, los mensajes y la conexión con Groq u Ollama, y LangGraph aporta el grafo del agente. | |

## 6. Un solo agente vs. sistemas multiagente

### 6.1 Un solo agente con herramientas

Un único LLM razona y decide qué herramientas usar. Las herramientas son funciones "tontas" (no piensan); toda la inteligencia está en el agente. Es la arquitectura más simple y suele ser suficiente cuando la tarea tiene un objetivo claro.

### 6.2 Sistemas multiagente

Varios agentes, cada uno con su propio LLM, instrucciones y herramientas, colaboran para resolver una tarea. Patrones comunes:

- **Supervisor:** un agente central recibe la tarea y delega en agentes especialistas.
- **Jerárquico:** supervisores que coordinan a otros supervisores, para tareas muy grandes.
- **Red o colaborativo:** los agentes se pasan el control entre sí según lo necesiten.

| Criterio | Un solo agente | Multiagente |
|---|---|---|
| Complejidad | Baja | Alta (coordinación y comunicación) |
| Llamadas al LLM | Pocas | Muchas |
| Costo y latencia | Menores | Mayores |
| Depuración | Sencilla | Difícil |
| Conviene cuando… | La tarea es acotada y las herramientas están bien definidas. | Hay subtareas grandes que requieren especialización o razonamiento independiente. |

> Regla práctica: empezar con un solo agente y pasar a multiagente solo cuando un agente con demasiadas herramientas o responsabilidades empieza a confundirse.

## 7. Aplicación en el proyecto

El "Agente para Compras Inteligentes" es **un solo agente** construido con LangGraph que sigue el patrón ReAct. El grafo tiene un nodo *agente* (el LLM gpt-oss-120b en Groq, o Llama local con Ollama), un nodo *herramientas* (`ToolNode`) y una arista condicional que decide si se usa una herramienta o se da la respuesta final. El estado es un `TypedDict` con la lista de mensajes.

Grafo del agente en LangGraph

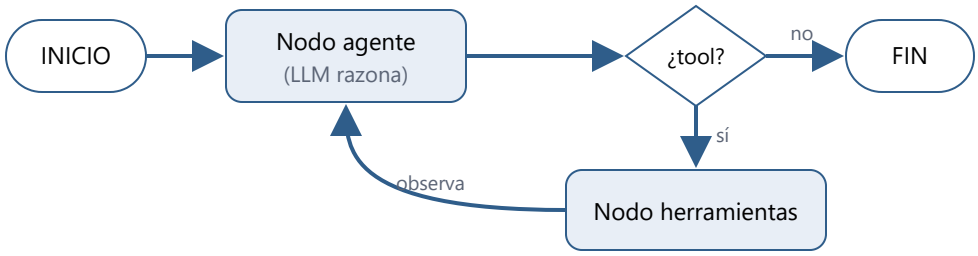



Figura 2. Grafo del Agente para Compras Inteligentes.

El agente cuenta con cuatro herramientas que corresponden a las acciones del proyecto: **buscador de productos** (opciones por tienda), **comparador de precios** (precio final, promedio y rango, calculados con Python), **analista de reseñas** (al menos tres opiniones y un score del 1 al 10) y **analista de usuarios relevantes** (personalidades que usan el producto). Se eligió un solo agente porque las cuatro acciones son operaciones concretas que no requieren razonamiento independiente: basta con que un agente decida cuándo usarlas e integre sus resultados en una recomendación. Un sistema multiagente habría aumentado el costo y la complejidad sin mejorar el resultado.

## 8. Conclusiones

- Un agente combina un LLM (razonamiento) con herramientas (acciones) y un ciclo de decisión. Esto le permite resolver tareas que un LLM solo no puede.
- El patrón ReAct hace que las respuestas se basen en datos observados, lo que reduce las invenciones del modelo.
- LangChain proporciona las piezas (modelos, mensajes, herramientas) y LangGraph organiza el flujo como un grafo con estado y ciclos. Son complementarios.
- Un solo agente con herramientas es la opción más simple, barata y fácil de depurar. Los sistemas multiagente se justifican solo en tareas grandes que requieren especialización.

## Referencias

1. Russell, S. y Norvig, P. *Inteligencia Artificial: Un enfoque moderno*. Pearson.
2. Yao, S. et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629.
3. Documentación de LangGraph. https://langchain-ai.github.io/langgraph/
4. Documentación de LangChain. https://python.langchain.com/

5. Documentación de Groq. https://console.groq.com/docs
6. Documentación de Ollama. https://ollama.com